Image Processing and Computer Vision (CSC6051/MDS5112) Assignment 2

Qiheng Peng
Student ID: 225040065
225040065@link.cuhk.edu.cn

Abstract

This report presents the implementations and experiments for Assignment 2, including Gaussian filtering (Task 1), global/local histogram equalization (Task 2), and monocular depth estimation on ScanNet (Task 3). For Task 1 and Task 2, I implemented all required operations in NumPy without using prohibited high-level APIs and visualized the qualitative effects. For Task 3, I trained a ResNet50-based baseline and evaluated it with aligned AbsRel, then performed a data-scale study under four training schedules, and finally compared with two foundation models (VGGT and Depth Anything 3). The results show that increasing training data while reducing epochs improved the baseline performance on the validation split in this setup, and foundation-model performance varied significantly depending on model/task alignment.

1. Task 1

Implementation Details I implemented Gaussian filtering from scratch in `task1/task1.py`. The key steps are:

- Input validation: checks for null input, odd kernel size greater than 1, and non-negative σ.
- Kernel generation: builds a 2D Gaussian kernel with

$$G(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right), \tag{1}$$

  then normalizes it so that the kernel sums to 1.
- Convolution: applies sliding-window weighted sum with `reflect` padding.
- Data type restoration: clips the output to $[0, 255]$ and converts back to original image type.

Both grayscale and RGB images are supported (channel-wise convolution for RGB).

Results & Analysis I tested with `kernel_size=5` and `sigma=1`. The output image is smoother and high-frequency noise/textures are reduced while keeping major structures intact.

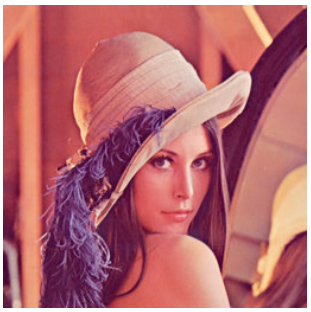

Figure 1. *

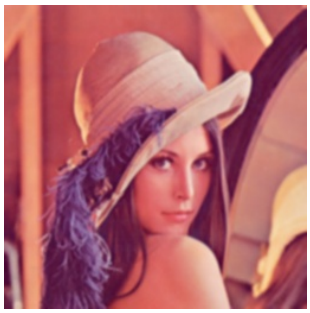

Figure 2. *

(a) Input (b) Gaussian result

Figure 3. Task 1 qualitative result of Gaussian filtering.

2. Task 2

Implementation Details I implemented both global and local histogram equalization in `task2/task2.py`.

Global histogram equalization.

- Compute histogram $h(i)$ for $i \in [0, 255]$.
- Compute CDF $c(i) = \sum_{j=0}^{i} h(j)$.
- Build LUT using

$$\hat{i} = \text{round}\left(\frac{c(i) - c_{\min}}{N - c_{\min}} \cdot 255\right), \tag{2}$$

  where N is the number of pixels and $c_{\min}$ is the first non-zero CDF value.

Local histogram equalization.

- Use a square window (31×31) centered at each pixel.
- Compute local histogram/CDF and map current pixel by local CDF.

- Use reflect padding for border handling.
- Use a sliding-window histogram update (subtract left column and add right column) to reduce repeated computation.

Results & Analysis Compared with global equalization, local equalization enhances local contrast more aggressively and reveals weak details in dark/flat regions. However, it may also amplify local noise and texture, which is consistent with adaptive histogram equalization behavior [3, 1].

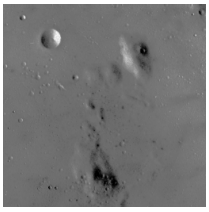

Figure 4. *
(a) Input

Figure 5. *
(b) Global HE

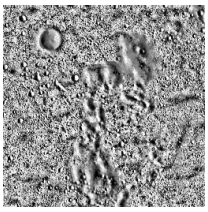

Figure 6. *
(c) Local HE

Figure 7. Task 2 comparison: local histogram equalization improves local details more strongly than global equalization.

3. Task 3

Setup and Protocol The training/evaluation pipeline is implemented in `task3/train.py` and `task3/test.py`. Following the assignment requirement, I trained the baseline model on ScanNet with the provided split files and reported aligned AbsRel [5, 4]. The default baseline uses a ResNet50 encoder-decoder depth network with SiLog loss.

For evaluation, predictions are first aligned to ground-truth depth using per-image scale-shift least squares, then AbsRel is computed on valid pixels. All quantitative results below come from the JSON logs in `task3/outputs/*`.

Baseline and Data-Scale Study The baseline setting is 32 epochs with 10k max training samples. I additionally trained four settings required by the assignment:

- 16 epochs + 20k samples,
- 8 epochs + 40k samples,
- 4 epochs + 80k samples,
- 2 epochs + 160k samples.

As shown in Fig. 10(b), increasing sample count under shorter epochs continuously improved validation AbsRel in this experiment (from 0.1158 to 0.0777), indicating that broader data coverage can be more beneficial than longer optimization on a smaller subset.

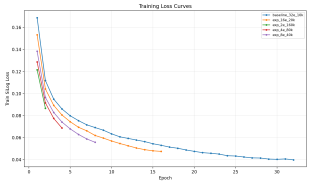

Figure 8. *
(a) Training loss curves

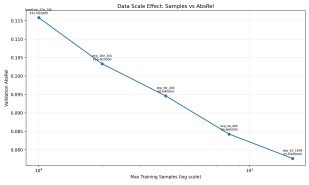

Figure 9. *
(b) Data scale effect

Figure 10. Task 3 training dynamics and effect of sample scale.

Foundation Model Comparison I evaluated two foundation models, VGGT [6] and Depth Anything 3 (DA3) [2], using the same evaluation protocol in `test.py`.

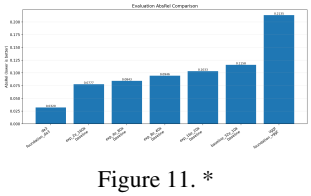

Figure 11. *
(a) Overall AbsRel comparison

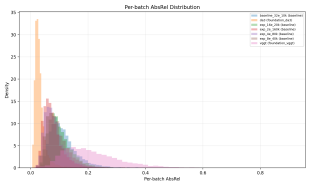

Figure 12. *
(b) Per-batch distribution

Figure 13. Task 3 quantitative comparison across baseline/data-scale/foundation methods.

Method	Setting	AbsRel (val)
Baseline	32e, 10k samples	0.1158
Baseline	16e, 20k samples	0.1033
Baseline	8e, 40k samples	0.0946
Baseline	4e, 80k samples	0.0843
Baseline	2e, 160k samples	0.0777
VGGT	foundation_vggt	0.2135
DA3	foundation_da3	0.0320

Table 1. Task 3 quantitative results (aligned AbsRel, lower is better).

In this setup, DA3 achieved the best AbsRel, while VGGT underperformed the baseline. A likely reason is that different foundation models have different preferred input formats and operating assumptions, and performance is sensitive to adaptation details in downstream evaluation scripts.

4. Conclusion

This assignment covers three levels of vision tasks: low-level filtering, contrast enhancement, and high-level depth estimation. Task 1 and Task 2 confirm that implementing classical methods from first principles helps understand how parameters and local statistics affect visual quality. In Task 3, the controlled experiments show clear data-scale trends for baseline training and highlight the impact of model adaptation when comparing with foundation models. The full pipeline (training logs, evaluation JSON, and plot-

ting scripts) is reproducible and can be extended for further benchmarking.

References

[1] Adaptive histogram equalization. https://en.wikipedia.org/wiki/Adaptive_histogram_equalization. Accessed: 2026-03-22. 2

[2] Depth anything 3: Recovering the visual space from any views. https://arxiv.org/abs/2511.10647. Accessed: 2026-03-22. 2

[3] Histogram equalization. https://en.wikipedia.org/wiki/Histogram_equalization. Accessed: 2026-03-22. 2

[4] Pytorch documentation. https://pytorch.org/docs/stable/index.html. Accessed: 2026-03-22. 2

[5] Pytorch tutorials: Learn the basics. https://pytorch.org/tutorials/beginner/basics/intro.html. Accessed: 2026-03-22. 2

[6] Vggt: Visual geometry grounded transformer. https://arxiv.org/abs/2503.11651. Accessed: 2026-03-22. 2